

PuzzleForge

An Agentic Puzzle Game Design and Generation System

Final Report

94-815 Agentic Systems Studio

Team: Yixiao Li, Kaizhen Tan, Hanzhe Hong

Track: A -- Technical Build

1. Problem Statement

Designing solvable puzzle games is difficult because it requires balancing creativity with correctness. A level can look plausible while still being impossible to solve. A single large language model cannot reliably verify Sokoban solvability; it can only guess. This creates a core design problem: creative generation and formal verification require different capabilities.

PuzzleForge addresses this problem by separating creative design from deterministic verification across five coordinated agents in a LangGraph pipeline. The system transforms a user's free-text game concept into a playable browser-based Sokoban puzzle game. The submitted final artifacts are two BFS-verified playable demos, one push game and one slide game, and every unresolved failure in the live evaluation runs is logged rather than hidden.

1.1 Target User and Decision Context

The target users are solo game designers, prototyping teams, and game-design students who want to turn a rough puzzle concept into a playable level set quickly. Their core decision is whether a generated puzzle is ready to show, needs repair, or should be rejected. PuzzleForge is built for that decision context: it generates a game, verifies each level with a deterministic solver, and exposes the trace, QA results, and failure reasons so users can inspect the result instead of trusting the LLM's claim.

2. Architecture

2.1 System Overview

PuzzleForge uses a two-layer architecture with five agents. The Design Layer contains the two creative agents: the Game Designer, which generates the overall game specification from a user concept, and the Level Designer, which creates and redesigns individual grid layouts. The Implementation Layer contains three agents responsible for verification and coordination: the Developer / Orchestrator, which translates designs into a structured configuration and routes failures using deterministic rules; the QA Tester, which uses a BFS solver to verify solvability, difficulty, and diversity; and the Debugger, which applies minimal tile-level fixes to broken configurations.

The guiding design principle is deterministic where possible, LLM where necessary. Solvability verification, routing classification, and iteration control are handled by code, not LLM judgment. LLMs are reserved for theme design, narrative framing, and spatial arrangement, while correctness is enforced by deterministic components.

Agent	Layer	Role	Tools	LLM
Game Designer	Design	Generate game spec from user concept	GPT-4o, temperature 0.8	Yes
Level Designer	Design	Create and redesign grid layouts	GPT-4o, temperature 0.7	Yes
Developer / Orchestrator	Implementation	Translate designs and route failures	Template engine, routing rules	No

QA Tester	Implementation	Verify solvability, difficulty, and diversity	BFS solver	No
Debugger	Implementation	Generate minimal config fixes	GPT-4o, temperature 0.3	Yes

2.2 Pipeline Flow

The pipeline runs in a forward pass followed by a conditional feedback loop. A user's concept flows through the Game Designer, the Level Designer, and the Developer before reaching the QA Tester. QA verifies every level with the BFS solver. If every level passes, the system finalizes and produces the playable game. If any level fails, the Developer classifies each failure and routes it either to the Debugger for a small configuration patch or back to the Level Designer for a full redesign. The fixed levels are then re-tested by QA. The loop terminates once all levels pass, the three-cycle cap is reached, the token budget is exhausted, or the five-minute wall-clock timeout fires.

```
User Input -> Game Designer -> Level Designer -> Developer (translate)
-> QA Tester (BFS solver) -> route_after_qa:
    all_pass      -> Finalize (playable game)
    has_failures  -> Developer (classify and route)
        config_bugs -> Debugger -> Apply Patches -> QA re-test
        design_flaws -> Level Designer -> Developer -> QA re-test
    max_cycles    -> Finalize with failures logged
    budget/timeout -> Finalize best available result
    generation_failed -> Finalize with failure status and trace
```

2.3 Orchestration Framework

The pipeline is implemented on top of LangGraph's StateGraph. A shared TypedDict holds pipeline state across nodes. Append-only fields such as trace logs and accumulated debug patches use Annotated[List, operator.add], so the system preserves history without manual bookkeeping. The graph consists of nine compiled nodes connected by five conditional edges that encode routing logic as explicit graph structure rather than hidden prompt instructions. This makes the control flow auditable: any path through the pipeline can be reproduced from the trace.

2.4 Routing Criteria

When QA reports failures, the Developer classifies each failure using ten deterministic rules implemented in src/orchestrator/routing.py. No LLM judgment is involved at this step. Five criteria identify configuration bugs and route the level to the Debugger. Five criteria identify design flaws and route the level back to the Level Designer.

ID	Criterion	Classification	Route
C1-C5	Placement issues, count mismatches, player on wall, direct config errors	Config bug	Debugger
D1-D5	Unsupported mechanics, grid too small, excessive structural changes, solver timeout, diversity violation	Design flaw	Level Designer

2.5 Stop Conditions

The pipeline terminates when any of five conditions is satisfied: all levels pass QA, three debug cycles have been exhausted, the 80K token budget has been exceeded, the five-minute wall-clock timeout has fired, or a generation step fails or fails schema validation after retry. This prevents infinite loops and prevents incomplete level sets from being presented as successful final games.

3. Implementation Details

3.1 BFS Sokoban Solver

The ground-truth verification tool lives in `src/solver/sokoban_solver.py`. It runs a breadth-first search over states represented as player position plus a frozen set of box positions. This guarantees that any returned solution is a minimum-move solution. The solver supports both push and slide mechanics. Push moves a box one tile at a time; slide moves a box until it collides with a wall or another box.

Three deadlock detectors -- corner, wall-line, and 2x2 freeze -- prune clearly unsolvable branches for push levels. Dead-square pruning is disabled for slide levels because slide physics changes what positions are reachable. A 500,000-state cap prevents runaway computation on pathological inputs. Solver timeouts are represented as `solvable=None` and route through criterion D4 instead of being reported as false unsolvable results.

Beyond solvability, the module exposes `estimate_difficulty()`, a one-to-five rating derived from solution length, states explored, and grid area, and `level_similarity()`, a Jaccard index used for diversity checks.

3.2 Inter-Agent Communication

The LLM-generated handoffs pass through Pydantic v2 models before downstream agents consume them. Invalid JSON triggers an automatic retry in which the validation error is fed back into the next prompt, giving the LLM specific guidance on what to fix. The key validated schemas are `GameSpec`, `LevelDefinition`, `DebugPatch`, `LevelConfig`, and `GameConfig`. QA reports and routing decisions are deterministic structured dictionaries, not LLM outputs; they are produced by code and preserved in the design document. The Level Designer validates that fresh generation returns exactly the requested level IDs; during redesign, it validates that the returned replacement IDs exactly match the levels selected for redesign. Partial level sets are retried or reported as generation failures instead of propagating silently.

3.3 Game Rendering

The HTML/JS template at `src/engine/templates/sokoban.html` renders playable games in the browser. It supports both push and slide mechanics. Five schema-supported color themes -- dark, earthy, icy, gothic, and space -- are driven by the `theme_details` block in `GameConfig`, giving each generated game a distinct visual identity. Entities are rendered as emoji chosen by the Game Designer, and the game header displays a mechanic badge so players understand the movement rule.

Two additions make the generated games more challenging. First, each level carries `min_moves` from the BFS solver. The UI displays Moves: X / limit and Best: N, where `limit = min_moves + 2`. If the player exceeds the limit without solving the level, the game transitions to a lost state and prompts a reset. Second, the submitted demos use non-trivial, solver-verified layouts: the push demo has five levels with shortest solutions of 15, 15, 24, 27, and 35 moves, and the slide demo has five levels with shortest solutions of 9, 13, 15, 21, and 26 moves.

4. Evaluation Results

4.1 Final Artifact and Live Pipeline Evaluation

The Phase 3 package uses two evidence layers. First, the submitted portfolio artifacts are the bundled push and slide demos in `outputs/demo_game.html` and `outputs/demo_slide_game.html`; both contain five BFS-verified levels and enforce a move limit derived from the solver's shortest solution. Second, four live pipeline runs stress-tested the agent loop on open-ended concepts across push and slide mechanics. These live runs reached terminal states and produced inspectable traces, design documents, and outputs. Two runs also missed the requested five-level contract, so that reliability issue is counted directly in the evaluation.

Run	Concept	Mechanic	Theme	Levels	Solvability	Rate	Cycles	Tokens	Time	Status
RUN-01	Space station cargo robot	push	space	5	4/5	80%	3	6,093	28s	completed_with_failures
RUN-02	Penguin sliding ice blocks	slide	icy	4	3/4	75%	3	8,538	44s	completed_with_failures
RUN-03	Gothic dungeon gem puzzle	push	gothic	5	5/5	100%	0	2,969	15s	completed
RUN-04	Zero-gravity curling	slide	space	3	2/3	67%	3	8,193	36s	completed_with_failures

Aggregated across the live runs, the overall solvability rate was 82% (14 of 17 generated levels), meeting the Phase 3 acceptance target of at least 80% post-QA solvability. Push solvability reached 90% (9 of 10), while slide solvability reached 71% (5 of 7), which identifies slide physics as the main remaining engineering challenge. Every live run reached a terminal status and produced inspectable output, for a 100% terminal output rate. Mean token usage was 6,448 per run, comfortably inside the 80K budget, and mean wall-clock time was 30.8 seconds. Debug cycles per run ranged from zero to three. The Game Designer correctly chose the mechanic from the concept in all four cases.

4.2 Success Criteria Assessment

Criterion	Target	Actual	Status
Overall post-QA solvability	>= 80% acceptance; >= 90% stretch	82% (14/17)	PASS
Push solvability diagnostic	>= 80% acceptance; >= 90% stretch	90% (9/10)	PASS
Slide solvability diagnostic	Track separately because slide physics is harder	71% (5/7)	GAP IDENTIFIED

Terminal output rate	$\geq 85\%$	100% (4/4 reached a final status with saved artifacts)	PASS
Latency	≤ 4 minutes	mean 30.8s	PASS
Token efficiency	$\leq 80\text{K}$ per run	mean 6,448	PASS
Routing accuracy	$\geq 75\%$	11/11 observed routing decisions in saved traces matched C2 or C4 criteria	PASS
Mechanic selection	Match concept	4/4 correct	PASS
Refinement lift	≥ 20 pp on a repair case	RUN-02 improved from 2/4 to 3/4 solvable (+25 pp)	PASS
Design diversity	Mean pairwise layout similarity ≤ 0.5	mean 0.28 across live runs; highest run mean 0.31	PASS
Visual theme variation	Distinct visual identity	4 narrative themes across 3 color families	PASS
Difficulty-rating curve	$\geq 80\%$ monotonic by QA difficulty rating	4/4 runs monotonic	PASS
Requested level count	5 levels per live run	2/4 runs met target	GAP IDENTIFIED

This framing separates acceptance criteria from diagnostic metrics. The system clears the overall Phase 3 acceptance threshold, while the slide-only result gives a concrete next target. Slide-mechanic levels are harder because the Level Designer must reason about boxes that keep moving until they hit a wall or another box; placement decisions have longer-range consequences than in standard push Sokoban. The live evaluation also shows a level-count reliability gap: RUN-02 produced four levels and RUN-04 produced three levels, both below the requested five-level contract.

4.3 Baseline Comparison

To demonstrate the value of the multi-agent architecture, we ran a single-LLM baseline: one GPT-4o call generated all five levels, with no solver verification, no debug loop, and no specialized agents. We then verified the baseline outputs with the same BFS solver used inside the pipeline. This comparison is diagnostic rather than a controlled benchmark: the baseline prompt asks for smaller 5x5-8x8 levels and up to 3 boxes, while the multi-agent pipeline prompts for more ambitious live-generation levels. The useful takeaway is not that the systems are perfectly matched; it is that solver-backed verification and repair create visible gains on the harder slide mechanic.

Metric	Single-LLM Baseline	Multi-Agent Pipeline	Interpretation
Push solvability	100% (10/10)	90% (9/10)	Comparable
Slide solvability	30% (3/10)	71% (5/7)	+41 percentage points
Overall solvability	65% (13/20)	82% (14/17)	+17 percentage points
Verified solvable	No	Yes	Critical difference
Tokens per run	1,348	6,448	Cost of verification and repair

The comparison produces four findings. First, slide mechanics expose a clear limit on LLM spatial reasoning: the baseline manages only 30% solvability on slide levels, while the pipeline reaches 71% with verification and repair. Second, push levels are relatively easy for LLMs, and both approaches achieve high solvability there. Third, the debug loop is the key differentiator: the pipeline spends more tokens per run, and this extra spend buys solver verification and iterative repair. Fourth, the baseline has no way to know which levels are broken; the pipeline catches every unsolvable level and either repairs it or logs it for downstream attention.

4.4 Test Cases Summary

We executed twelve scenario-level test cases, recorded in `eval/test_cases.csv`. All twelve passed. The cases cover one end-to-end happy path, solvability-loop repair, difficulty-curve detection, design-flaw routing, soft diversity handling, layout-similarity validation, Pydantic validation, Debugger escalation, solver-timeout routing, final artifact verification, and slide-mechanic selection.

5. Failure Analysis

5.1 Failures Discovered and Fixed

We discovered and resolved several issues during live Phase 3 testing. The three most instructive failures are summarized below; all eight documented failures appear in `eval/failure_log.md`.

F-004: Silent Redesign Failure

The Level Designer's `_redesign_levels()` function originally caught all exceptions with a broad `except` block and returned an empty redesign. When the LLM returned invalid JSON during a redesign pass, the output was silently discarded. The broken level was never replaced, so the pipeline ran all three debug cycles without making progress.

The fix was to replace the blanket exception handling with retry logic that re-invokes the LLM using the specific validation error as feedback. The redesign prompt now includes original level dimensions and solver-timeout guidance. The governance lesson is that silent failure in a feedback loop is worse than a crash: the system appeared to be working while doing nothing useful.

F-005: Diversity Hard-Blocking

When all five levels were solvable but one pair exceeded a Jaccard similarity threshold, the pipeline treated the diversity violation as a hard failure. It burned all three debug cycles trying to fix a cosmetic issue, even though the Debugger only adjusts tile positions inside a single level and cannot fix whole-set diversity.

The fix introduces a soft-fail path: if every level is already solvable and at least one debug cycle has been attempted, the pipeline accepts the result and finalizes. Diversity is now treated as a soft constraint. The lesson is that not all failures are equal; a loop that treats cosmetic issues with the same urgency as critical bugs wastes its iteration budget.

F-006: Debugger Processing Stale Levels

The Debugger originally computed `levels_to_fix` by combining current failures with accumulated routing decisions. Because `routing_decisions` is append-only, levels repaired in cycle 1 reappeared in later cycles. The root cause was a mismatch between state pattern and consumption pattern: append-only history is valuable for observability, but dangerous for current-cycle decision-making.

The fix changes the Debugger to read only `failed_level_ids`, which is reset each cycle by `developer_route_node`. The governance lesson is that agents that act on state must clearly distinguish audit records from current instructions.

This fix is reflected in the current Debugger and patch-application code path: the Debugger reads the current-cycle `failed_level_ids`, and the Developer applies only the most recent patch per still-failing level. The submitted live-run design documents still preserve append-only routing and patch histories, so repeated level IDs remain visible there as evidence of the earlier failure pattern rather than as current-cycle instructions.

5.2 Failure Patterns

Across all eight documented failures, three patterns emerge. First, silent failures are the most dangerous class of bug. F-003 and F-004 both involved invalid LLM output, but F-003 was caught explicitly by Pydantic and recovered through a transparent retry, while F-004 was silently swallowed. Second, feedback loops need severity awareness. F-005 showed that a loop without tiered severity can spend its entire retry budget on low-priority problems. Third, append-only state requires careful consumption. F-006 showed that the same pattern that enables strong observability can produce incorrect behavior when agents read accumulated history as current instructions.

6. Governance Reflection

6.1 Governance Controls and Effectiveness

Control	Implementation	Effectiveness	Evidence
BFS solver as ground truth	QA Tester uses deterministic solver for push and slide	Highly effective	14/17 levels verified solvable across four live runs
Deterministic routing	10 formal criteria in <code>routing.py</code>	Effective	No observed routing misclassification
Debugger bounded authority	25% tile-change threshold, temperature 0.3	Effective	Escalation correctly triggered when needed
3-cycle iteration cap	Hard limit in <code>route_after_qa()</code>	Effective but blunt	Prevents infinite loops but does not differentiate severity
80K token budget	Checked at conditional edges and in <code>call_llm()</code>	Effective	Never hit in practice
Pydantic validation	LLM-generated schemas validated before downstream use	Effective	Caught invalid outputs and enabled retry
Append-only trace logging	Annotated[<code>List</code> , <code>operator.add</code>]	Strong for observability	Full history preserved, but required careful consumption

6.2 What We Would Change

With the benefit of hindsight, three changes stand out. The three-cycle cap should become severity-aware, allocating cycles based on failure type. Solvability failures should take priority, and diversity issues should get at most one attempt. Exception handling should be structured rather than broad; every exception handler should retry with feedback, log explicitly, or escalate rather than silently continue. Finally, append-only state should be kept separate from decision state. routing_decisions is valuable as an audit trail, but current-cycle fixes should consume a fresh field that is reset each cycle.

6.3 Human-in-the-Loop

The current system is fully automated from concept to finished game. The human provides the initial concept, receives the completed game and trace log, and can inspect every agent decision through the Streamlit dashboard. For production deployment we would add three oversight hooks: a routing override that lets a human reclassify a failure, a level editor that allows manual adjustment of generated layouts, and approval gates before finalizing.

7. Lessons Learned

7.1 Technical Lessons

LangGraph's reducer pattern is powerful but requires discipline. Annotated[List, operator.add] works well for trace accumulation, but any agent that acts on accumulated state must distinguish history from current instructions. Retry logic is essential for LLM-based agents: Game Designer, Level Designer, and Debugger can all produce invalid JSON, and retry-with-error-feedback is both cheap and effective.

The BFS solver is the system's most reliable component. It never relies on LLM claims, and grounding agent output in deterministic verification became the strongest governance mechanism in the system. Finally, soft and hard constraints must be explicit. F-005 showed that conflating "must fix" with "nice to fix" can exhaust the iteration budget on the wrong problems.

7.2 Process Lessons

Live testing reveals bugs that unit tests miss. The most important Phase 3 bugs passed isolated tests but failed during live pipeline execution because the interaction between agents exposed issues that isolated testing did not. Observability paid for itself: the trace log made it possible to diagnose silent redesign failure, diversity hard-blocking, and stale-level processing.

8. Individual Contribution Reflection

Yixiao Li

My responsibilities were the Game Designer agent, the Level Designer agent, and the BFS solver. During Phase 3 I discovered and fixed F-004 by adding retry with error feedback and improving the redesign prompt, F-005 by adding a soft-fail path to the graph, and F-006 by correcting the Debugger's state consumption. I ran the four live pipeline tests, collected evaluation data, and wrote the failure analysis and evaluation sections. I also kept `test_cases.csv`, `evaluation_results.csv`, `failure_log.md`, and `version_notes.md` in sync with code changes.

The most important thing I learned is that multi-agent systems fail in the interactions between agents, not only inside individual agents. Each agent worked correctly in isolation, but the way they shared state and responded to each other's outputs produced emergent bugs. The append-only state issue behind F-006 was especially instructive: a pattern that is excellent for logging caused incorrect behavior when used for decision-making.

Kaizhen Tan

My responsibilities were the QA Tester agent, Debugger agent, HTML/JS game template, and Streamlit dashboard. During Phase 3 I tightened the QA path so generated levels are judged by the deterministic BFS solver rather than LLM claims, exposed `min_moves`, `states_explored`, `timeout`, and failure reasons in the design document, and kept the UI aligned with the evidence package. I also worked on move-limit enforcement in the browser game and failure-case visibility in the dashboard, so a reviewer can inspect why a level passed, failed, timed out, or needed another cycle. In addition, I checked the demo and evaluation screens against the saved trace files to make sure the UI reported the same solver outcomes, debug-cycle status, and boundary cases as the evidence package.

The most important thing I learned is that an agentic demo is only credible when the evidence layer is visible to the user. A playable game matters, but the QA report, trace viewer, and boundary behavior are what make the system auditable. This changed how I think about front-end work in agentic systems: the interface should not only show the result, it should also expose the reasoning, verification, and limits behind the result.

Hanzhe Hong

My responsibilities were the Developer / Orchestrator layer, LangGraph pipeline, routing logic, and coordination between agents. During Phase 3 I implemented the deterministic handoffs from design to translation to QA, maintained the routing criteria that separate configuration bugs from design flaws, and kept the pipeline bounded with stop conditions, token budget checks, and max-cycle controls. I also helped diagnose the feedback-loop failures where redesign, debugging, and accumulated state interacted in unexpected ways.

The most important thing I learned is that multi-agent architecture needs explicit contracts between components. Clear state ownership, reset points, and routing rules matter more than adding more agents. Without those contracts, a system can look modular while still passing stale or ambiguous instructions between roles.

9. Evidence Package Summary

The evaluation and failure packages are shipped alongside the code. We executed twelve scenario-level test cases in `eval/test_cases.csv`: one clean happy-path run, final-artifact verification, a solvability-loop deadlock fix, difficulty-curve detection, design-flaw routing, live debug loop behavior, soft diversity handling, layout-similarity metric validation, Pydantic validation, Debugger escalation, solver-timeout routing, and slide-mechanic selection. All twelve passed.

Eight failure cases are documented in `eval/failure_log.md`, covering corner deadlocks, similar layouts, invalid mechanics, silent redesign, diversity hard-blocking, stale-level processing, slide-solvability limitations, and partial level-set generation. Pipeline run data, including the baseline comparison, lives in `eval/evaluation_results.csv`. The repository also includes trace files, design documents, sample outputs, screenshots, and the submitted 5-minute video walkthrough link.

10. Future Improvements

Six improvements would meaningfully strengthen the system. First, a severity-aware iteration budget would allocate debug cycles based on failure severity rather than treating all failures equally. Second, difficulty-curve enforcement during redesign would feed the full level set's difficulty ratings into the redesign prompt, helping maintain monotonic progression. Third, additional puzzle mechanics such as switches or teleports would extend the solver and template beyond push and slide.

Fourth, human-in-the-loop routing overrides would let users reclassify a failure from the Streamlit dashboard before the next debug cycle. Fifth, parallel level generation would replace the current single-call strategy, reducing the impact of one invalid level on the batch. Sixth, solver-guided level design would feed solver metrics back to the Level Designer during initial generation, not only during redesign.

Appendix: Repository Structure

```
PuzzleForge/
+-- README.md           # Project overview and setup
+-- AI_USAGE.md         # AI tool usage disclosure
+-- requirements.txt     # Python dependencies
+-- .env.example        # Environment template
+-- docs/
|   +-- Final Report.pdf # Phase 3 final report
|   +-- final_report.md  # Markdown source for final report
|   +-- architecture_diagram.pdf # System architecture diagram
|   +-- PuzzleForge_Executive_Summary.pptx # Executive summary deck
|   \-- screenshots/    # Screenshots with index
+-- media/
|   \-- demo_video_link.txt # 5-minute video link
+-- src/
|   +-- models.py        # Pydantic data models
|   +-- state.py         # LangGraph state schema
|   +-- config.py        # Configuration and thresholds
|   +-- main.py          # CLI entry point
|   +-- app.py           # Streamlit dashboard
|   +-- agents/          # 5 agent implementations
|   +-- solver/          # BFS Sokoban solver
|   +-- engine/          # HTML/JS game template engine
|   +-- orchestrator/    # LangGraph pipeline and routing
```

```
|  \-- utils/                # Trace logger, LLM wrapper
+-- eval/
|  +-- test_cases.csv        # 12 test scenarios with results
|  +-- evaluation_results.csv # Pipeline and baseline run data
|  +-- evaluation_plan.md    # Evaluation methodology
|  +-- failure_log.md        # 8 documented failures
|  +-- baseline.py           # Single-LLM baseline script
|  +-- baseline_results.json # Baseline run results
|  \-- version_notes.md     # Version history
+-- outputs/
|  +-- demo_game.html        # Final push demo
|  +-- demo_slide_game.html  # Final slide demo
|  +-- run1/ run2/ run3/ run4/ # Live test run evidence
|  \-- sample_traces/        # Sample configs and sample trace
\-- tools/
    +-- build_report_pdf.py   # Markdown-to-PDF report builder
    \-- build_report_docx.py  # Executive summary validation helper
```